Lab 5 – Shortest Path Algorithms on A Weighted TileMap [100 Marks]

A Partially Autograded Evaluation with Project Integration

IMPORTANT: **Your submission must compile without syntactic error to receive grades. Non-compilable solutions will not be graded.**

IMPORTANT: **You must upload your TileGraph.txt to *Avenue Dropbox* before the deadline to receive grades.**

Lab Deadline

The deadline for Lab 5 is Saturday, April 4, 2026, at 11:59 pm.

Deliverables

1. **You must upload on or before the posted deadline the following items to the Avenue dropbox:**
   - TileGraph.txt (i.e. change .java extension to .txt)
   - You further will demonstrate your Model Code and the correct automatic navigation behaviour for the tracer object.

Premise

Labs 4 and 5 of the Winter 2025 COMPENG 3SM4 are two branches of an automatic pathfinder AI to be used in the vintage classic top-down 2D sandbox game like Grand Theft Auto 1 / 2 from the 90s. Your software team is responsible for designing simple AI logics of an automated vehicle that will always try to navigate in a city map and find the best route from the starting intersection to the "goal" intersection to complete the mission. With Lab 4 out of the way, here is the updated scope of the remaining Project, with a major change in the project direction.

Lab 5 – Automated Path Optimization using Different Shortest Path Algorithms with Score Penalty and Rewards

> In this lab, the software team further optimizes the game by introducing the score penalty and reward pickups along every path in the map. Three different path optimization algorithms are to be deployed as three AI modes to deal with different map configurations. Using your provided algorithm implementations, the game development team will pick the most suitable one for the final prototype implementation.

The end result of these two labs will be a collection of simple AI scoring logic modules that can be interchangeably deployed in the automatic driver AI, as well as a prototype of multi-stage gameplay.

Objective Overview

Lab 5 focuses on the implementation of a *Weighted Directed Graph* from an updated TileMap, now with randomized score penalty and reward pickups spawned anywhere on the paths in the map. With this update, you are tasked with updating your TileGraph class with Weighted Edges, then implementing three different shortest path algorithms designed to deal with different weighted directed graph configurations.

The scope of the lab is the following:

- Implementation of the Weighted Graph of Tiles using the Adjacency List architecture
    - Each item in the adjacency list of a Tile vertex is now an object of the **WeightedEdge** class. Each such object stores the destination Tile and the weight of the edge.
    - The implementation of the **WeightedEdge** class is provided.
- Implementation of the *Topological Sort Algorithm* to support the Shortest Path Algorithm on Direct Acyclic Graphs (DAGs).
- Implementation of the *DAG Shortest Path Algorithm*.
- Implementation of *Dijkstra's Shortest Path Algorithm* for Nonnegative-Weighted Directed Graphs.
- Implementation of *Bellman-Ford Shortest Path Algorithm* for General Weighted Directed Graph.
    - Also to detect if negative weight cycles are present in the graph.
- Upgrading the buildGraph() method to take into account the weight of each edge by accounting for the Score Penalty and Score Reward pickups along each edge.

Coding Objectives

Analyze and understand the updated objects in the model code – Tile class, TileMap class, Tracer class, and the main model code.

Use the provided test bench **in the same class**; complete all the missing implementations of TileGraph.java.

You may copy your code from Lab 4 as a starting point for some of the implementations. Pay attention to the updates that are needed since now each adjacency list contains WeightedEdge objects instead of Tile objects.

Model Code Integration Objectives

Once you have implemented the required methods and passed all the test cases, you need to complete the model code integration by updating the buildGraph() method in the TileGraph class as follows:

- You need to update the buildGraph() implementation from Lab 4 by taking into account the following additional in-game pickups along the way:
    - Score Penalty (ASCII 'x') Worth 5-point penalty if picked up
    - Score Reward (ASCII '$') Worth 2-point reward if picked up
- Using the above two new elements, your buildGraph must scan through the map to construct the Weighted Graph.
- The objective of the game is to find the path with the largest overall score, i.e., the path for which TotalReward – TotalPenalty is the largest. This is also the path for which TotalPenalty – TotalReward is the smallest.
- This objective can be achieved by solving the weighted shortest path problem. For this, the weight of each edge has to be computed as the total penalty minus the total reward.

After completing buildGraph(), you should be able to see the path automatically generated and drawn on the TileMap in the game, and the tracer (P) will automatically follow the generated path to travel to the Goal intersection from the Start intersection.

Marking Scheme

- **[Total 80 Marks] Autograding Section – TileGraph Code Implementation and Behaviour**
  - **[10 Marks]** Implement the new AddEdge() method with weighted edge – 4 marks
    Pass both testAddWeightedEdge test cases – 4 marks
    Create and pass your own testAddWeightedEdgeCustom() test case – 2 marks
  - **[10 Marks]** Implement topologicalSort() method – 4 marks
    Pass both testTopologicalSort test cases – 4 marks
    Create and pass your own testTopologicalSortCustom() test case – 2 marks
  - **[14 Marks]** Implement DAGShortestPath() method – 5 marks
    Pass both testShortestPathDAG test cases – 4 marks
    Create and pass your own testShortestPathDAGCustom() test case – 2 marks
    Pass another test case written by the TA – 3 marks
  - **[20 Marks]** Implement DijkstraShortestPath() method – 10 marks
    Pass both testShortestPathDijkastra test cases – 4 marks
    Create and pass your own testShortestPathDijkstraCustom() test case – 2 marks
    Pass another test case written by the TA – 4 marks
  - **[26 Marks]** Implement BellmanShortestPath() method – 12 marks
    Pass both testShortestPathBellmanFord test cases – 4 marks
    Pass both testNegativeCycleBellmanFord test cases – 4 marks
    Create and pass your own testShortestPathBellmanFordCustom() test case – 2 marks
    Pass another test case written by the TA – 4 marks

  - **Note:** If test case failed, the test case marks are deducted, and only partial marks will be awarded to the implementation marks based on the severity of the logic error.

- **[Total 20 Marks] Lab Demo Section – Updated Four Shortest Path Test Modes**
  - **[10 Marks]** **(Code Walkthrough)** Demonstration of the updated buildGraph algorithm that generates the weighted directed graph. The algorithm must use the provided WeightedEdge class.
  - **[10 Marks]** Demonstration of the resultant Automatic Path Generation and Tracer (P) following the generated path from the Start intersection to the Goal intersection to deliver the following:
    - For Maps of Type B:
      The path with the least number of score penalty pickups (x).
    - For Maps of Type C
      The path with the highest overall score.

System Overview

With the game system upgraded, the game now offers three different TileMap configurations as outlined below (note that for each configuration, the Tracer automatically chooses the suitable shortest path algorithm):

A) Plain Map (No Pickups)

Original map type from Lab 4, where the shortest path is found by passing through the least number of intersections (I).

```
#########################################
#S I  I           I     I        I######
#.# ## ########### ##### ######## ######
#.# ##I           I     I######## ######
#.# ## ########### ##### ######## ######
#.# ## ########### ##### ######## ######
#.#I  I           I  I  I        I    I#
#.# ## ############## ## ######## #### #
#.# ## ############## ## ######## #### #
#I.I  I              I  I        I#### #
###.################## ########### #### #
###.################## ########### #### #
###.################## ########### #### #
###I................I...........I..P.D#
#########################################
```

B) Penalty-Only Map

The second map type, where every path in the map has its corresponding number of score penalty pickups. The shortest path in this map type is the path that picks up the least number of score penalties ('x').

<u>Suitable Shortest Path Algorithms</u>

Dijkstra's Algorithm and Shortest Path in a DAG with Topological Sort

```
#########################################
#S.I.x....x..x.......I...P...xI xI######
#x# ################x########x## ######
# # ##################I   x   xI..I....I#
# # ################## ######## ## ####.#
# #x################## ######## ## ####.#
# # ##################I  I     Ix I####.#
# # ################## ## ##### ## ####.#
#x# ################## ##x#####x## ####.#
#IxI x x       x     I  I    xI## ####.#
### ################## ## ##### ## ####.#
### ################## ## ##### ## ####.#
### ################## x## ##### ##x####.#
###I     x         x I  I x   Ix I  x D#
#########################################
```

Score Penalty 'x'

C) General Pick-Up Map (Score Reward and Penalty Mixed)

The third map type, where every path in the map sees a mixture of score penalty ('x') and reward ('$') pickups, resulting in a possibility of negative-weighted edges in the graph.

Suitable Shortest Path Algorithm

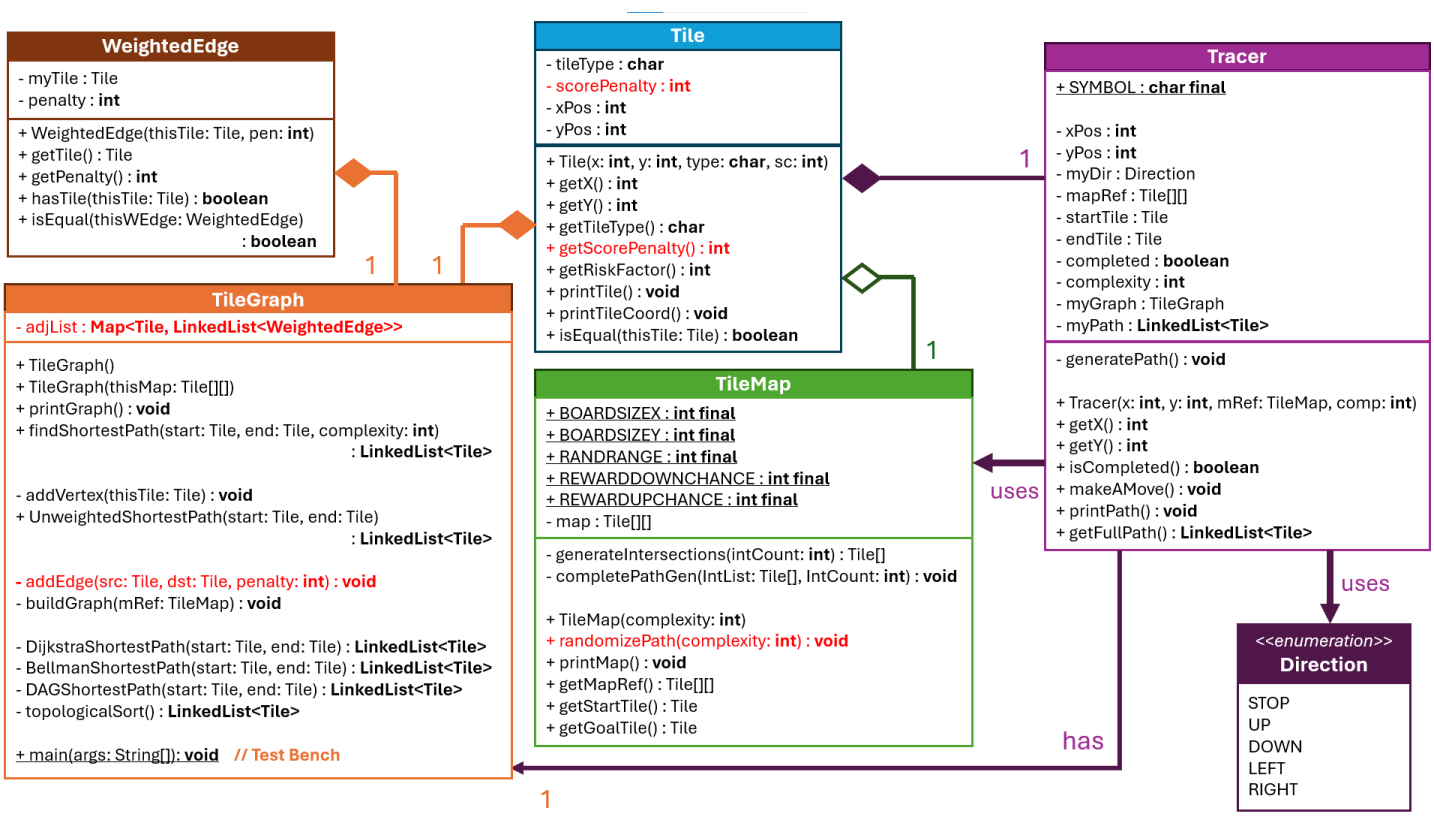
The Bellman-Ford Algorithm and Shortest Path in DAG with Topological Sort

```
#######################################
#S.......x$....I..I.$..$I###############
#$############## ## #####.###############
#I I xIx       I  Ix$  xI###############
#x# ## ######## ## #####.###############
# # ## ######## ## #####.###############
#I Ix I########I $I x   I..P..I   x$  I#
# #$## ######## ######## #####.####### #
# # ## ######## ########x#####.####### #
#I I  I        I########I    $I....$..I#
###### ######## ######## ##### #######.#
###### ######## ######## ##### #######x#
###### ######## ######## #####x#######.#
######I     x $I  xx    Ix    I   x x$D#
#######################################
```

Score Reward '$'

In modes B and C, the score penalties are considered positive weights to the edges, and the score rewards are negative weights. For prototyping purpose, the starting intersection (S) remains on the top-left corner of the map, and the goal/destination intersection (D) is always on the bottom-right corner. The Tracer (P) deploys different self-driving algorithms under different map modes, so to generate the optimal navigation path to travel from the S to D.

The following UML diagram summarizes the updated software design. Updates are highlighted in **RED**.

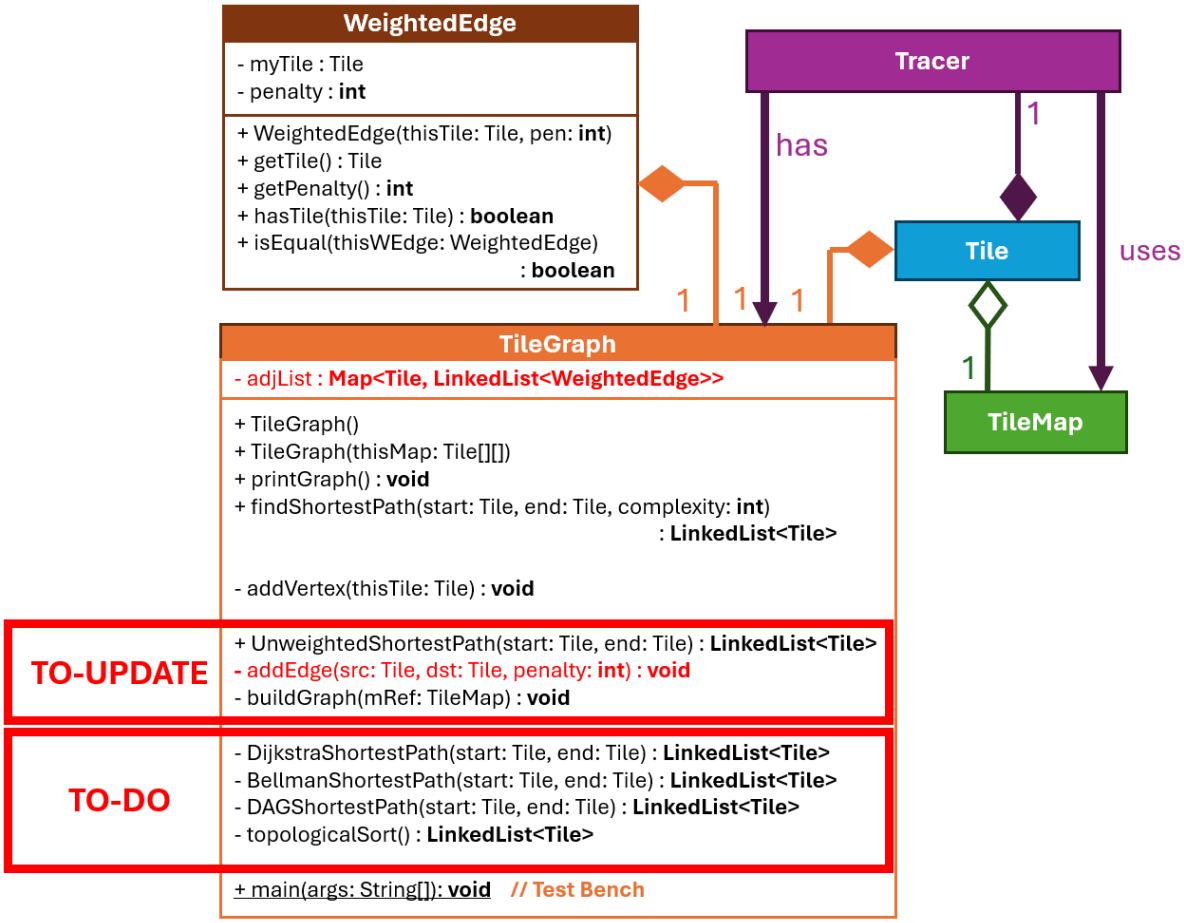


Your tasks are to complete the updated TileGraph implementations with the new private WeightedEdge class defined in TileGraph.java, and thoroughly test it using the provided test bench. Furthermore, you should update the buildGraph() method with the new WeightedEdge definition, so to ensure that the generated graph from the map is Weighted.

Weighted TileGraph – Design Specifications

This section summarizes the design specifications of the new TileGraph class. All relevant and helper methods from other classes are highlighted here as well.

All methods that require your implementation will be tagged **[TO-DO]** in the skeleton code.



I. TileGraph Members (Notice the Highlighted Updates)

- **private Map<Tile, LinkedList<WeightedEdge>>** adjList
  - A map collection of pairs (Tile, LinkedList), where the first member is a vertex and the second member is the adjacency list of that vertex. The adjacency list is a linked list of WeightedEdge objects.

- **public** TileGraph()
  - Default constructor of TileGraph; instantiates an empty graph

- **public** TileGraph(TileMap thisMap)
  - Constructor; instantiates a graph representing the input map of tiles
  - It invokes the method buildGraph () (which you have to implement) to populate the graph

- **private void** addVertex(Tile thisTile)
  - Adds thisTile as a new vertex in the adjacency list, if it is not already there.

- **public void** printGraph()
  - Prints a representation of the graph.

- **public LinkedList<Tile>** findShortestPath(Tile start, Tile end)
    - This is a façade function that selects the appropriate shortest path algorithms based on the type of the map.
    - The implementation is provided.

- **[TO-UPDATE] private void** addEdge(Tile src, Tile dst, **int penalty**)
    - Update this method from Lab 4 so that it adds a ***weighted directed*** edge (WeightedEdge) from the source Tile (src) to the destination Tile (dst).
    - If the designated WeightedEdge already exists, **do not add**.
    - Any new edge should be added at the end of the list.

- **[TO-UPDATE] private void** buildGraph(TileMap mRef)
    - Update this method from Lab 4 so that it adds vertices and ***weighted edges*** to the existing empty graph to build the graph corresponding to mRef
    - Vertices are intersections; edges are roads connecting two consecutive intersections from left to right or from up to down
    - Edges are directed and weighted: from up to down and from left to right. For each edge, count the number of score pickups along each scanned path
        - Score Penalty (x) adds a +5 weight to the edge
        - Score Reward ($) adds a -2 weight to the edge
        - One scanned edge may have more than one score pickup. You need to sum their overall net weight, and use it as the weight of the edge.
            - For example, if you encountered 2 x's and 1 $ for one scanned edge, the weight of this edge is 5 + 5 – 2 = 8.

- **[TO-UPDATE] public LinkedList<Tile>** UnweightedShortestPath(Tile start, Tile end)
    - Update the method findShortestPath() from Lab 4 to work in the updated TileGraph class. Intended to be used for **Map Type A (Plain Map).**
    - The vertices on the path have to appear in reversed order in the list (from end to start).

- **[TO-DO] private LinkedList<Tiles>** DijkstraShortestPath(Tile start, Tile end)
    - Implement the Dijkstra's shortest path algorithm to find the shortest path from start to end.
    - The shortest path is returned as a Linked List of Tiles from the end tile back to the start tile (reversed order, as in lab 4).
    - Intended to be used as one of the options for **Map Type B** (for this Map Type, there are no reward pickups, therefore the shortest path is the path with the smallest number of score penalty pickups).

- **[TO-DO] private LinkedList<Tiles>** DAGShortestPath(Tile start, Tile end)
    - Implement the faster shortest path algorithm to find the shortest path from start to end in a DAG.
    - The method should return the list of Tiles on the shortest path in reversed order (i.e., starting with the end Tile and going backwards).
    - Intended to be used as one of the options for **Map Types B and C.**

- **[TO-DO] private LinkedList<Tiles>** topologicalSort()
    - Implements the topological sort algorithm discussed in class, and returns the ordered list of Tiles.
    - Used by the method DAGShortestPath().

- **[TO-DO] private LinkedList<Tiles>** BellmanShortestPath(Tile start, Tile end)
    - Implements the Bellman-Ford shortest path algorithm to find the shortest path from start to end.

- The method returns the list of Tiles on the shortest path in reversed order (i.e., starting with the end Tile and going backwards).
- If negative weight cycles are detected, it returns null.
- Intended to be used as one of the options for **Map Type C (Direct Acyclic Graph with General Weights)**

II. WeightedEdge Class

Provided, but you need to know how to use this class to complete the required methods in the updated TileGraph implementation.

- **private** Tile myTile
  - The destination Tile of this directed edge.

- **private int** penalty
  - The weight of this directed edge.

- **public** WeightedEdge(Tile thisTile, **int** pen)
  - Constructor. Initializes the weight (penalty) and the destination Tile (myTile).

- **public** Tile getTile()
  - Getter. Returns the destination Tile of this WeightedEdge.

- **public int** getPenalty()
  - Getter. Returns the weight of this WeightedEdge.

- **public boolean** hasTile(Tile thisTile)
  - Checking whether this WeightedEdge points to the destination Tile that equals thisTile.

- **public** isEqual(WeightedEdge thisWEdge)
  - Checking whether thisWEdge and the current WeightedEdge instance are identical in weight and in the destination Tile.

III. Useful Java Container Classes

Using the existing Java container classes is allowed in this lab, such as:

- Map<K,V> / HashMap<E>
- Set<E> / LinkedHashSet<E>, HashSet<E>
- Stack<E>
- List<E> / LinkedList<E>
- PriorityQueue<E>
- Collection<E>
- Iterator<E>

They are defined in package java.util in module java.base. The documentation can be found at: https://docs.oracle.com/en/java/javase/11/docs/api/java.base/java/util/package-summary.html

Example: If you need a linked list of Tile objects in your code, you can use:

LinkedList<Tile> list = new LinkedList<Tile>(); // this line declares the reference variable "list", instantiates a LinkedList object and makes "list" point to this object.

Now "list" is a reference to an empty list of Tiles. You can add items to the list or perform other operations on the list by using methods from the class LinkedList<E>.

IV. Useful Tips

You have the option to define other private methods as helper methods in the TileGraph class, as well as additional inner classes.